# 🧠 Neural Networks Study Guide: Lecture 1 - Part 2 (Perceptron Learning & Delta Rule)

This study guide details the classification boundaries, learning algorithms, trace tables, and mathematical update rules for Single-Layer Perceptrons (SLP) using both the classical **Perceptron Learning Algorithm** and the **Delta Rule**.

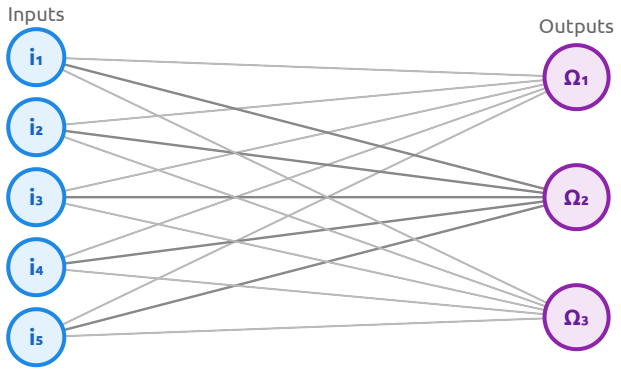
---

## 1. Single-Layer Perceptron (SLP) & Decision Boundaries

A Single-Layer Perceptron maps inputs directly to one or more output units to perform linear classification.

### Architecture with Multiple Outputs

When a classification task involves multiple output classes, inputs are mapped directly to multiple output units ($\Omega_1, \Omega_2, \Omega_3$).




### Decision Boundary Dimensionality

The geometric shape of the decision boundary is directly determined by the dimensionality of the input feature space:

1. **1D Space (Single Feature $x$)**:
   - The decision boundary is a **single point** along the number line.
   - **Linearly Separable Case**:

     ```
     Number Line:  ---[ Blue (-0.5) ]---[ Blue (1.1) ]---[ Blue (1.2) ]---[ Blue (2.6) ]---| Boundary Point |---[ Red (2.8) ]---[ Red (3.4) ]---[ Red (4.0) ]--->
     ```

     A single threshold point splits the classes.
   - **Non-Separable Case (Need Multiple Boundaries)**:

    - Pattern `Blue, Blue, Red, Blue`: Requires at least 2 separator boundaries; a single-layer perceptron (SLP) cannot represent this.
    - Pattern `Blue, Blue, Blue, Red, Blue, Red, Red`: Requires at least 3 separator boundaries; cannot be solved by an SLP.
2. **2D Space (Two Features $x_1, x_2$)**:
    - The decision boundary is a **straight line**.
    - **Linearly Separable Dataset Example**:
        - *Target 0 (Blue)*: $(2, 3), (3, 4), (7, 2)$
        - *Target 1 (Red)*: $(-3, 3), (-1, 2), (0, 1), (0, -2)$
        - *Solution*: A single diagonal line separating the red cluster on the left from the blue cluster on the right.
    - **Non-Linearly Separable Dataset Example** (Adding coordinates $(3, 8)$ and $(7, 5)$ as Target class 1):
        - Adding these points forces the red and blue clusters to overlap, making linear separation impossible; an SLP cannot resolve this.
3. **3D Space (Three Features)**:
    - The decision boundary is a **2D flat plane**.
4. **Higher Dimensional Space ($N > 3$ Features)**:
    - The decision boundary is an $(N - 1)$**-dimensional hyperplane**.

---

# 2. The Perceptron Learning Algorithm

The Perceptron Learning Algorithm is a supervised learning method for binary classifiers. It adjusts connection weights only when the network makes an incorrect prediction.

## Algorithm Pseudocode

```
While there exists a training pattern p in P where the error is too large:
    Input pattern p into the network and compute output y
    For each output neuron Ω:
        If y_Ω == t_Ω:
            Output is correct; do nothing
        Else:
            If y_Ω == 0:
                For all input neurons i:
                    w_{i,Ω} := w_{i,Ω} + o_i   (Increase weight by input
value)
            If y_Ω == 1:
                For all input neurons i:
                    w_{i,Ω} := w_{i,Ω} - o_i   (Decrease weight by input
value)
```

Where:

- $y_\Omega$: Calculated network output for unit $\Omega$.
- $t_\Omega$: Target (ground-truth) output class ($0$ or $1$).
- $o_i$: Input value from neuron $i$.

## Decision Rule

The prediction is determined by:

$$y = \begin{cases} 1 & \text{if } net \geq \theta \\ 0 & \text{if } net < \theta \end{cases}$$

Where $net = w_1 x_1 + w_2 x_2 + w_{\text{bias}} \cdot \text{bias}$, and $\theta$ is the threshold.

## Weight Update Trace Table (Step-by-Step)

Given initial weights $w_1 = 0.1, w_2 = 0.2, w_{\text{bias}} = -0.2$, inputs $x_1, x_2$, bias value $1$, threshold $\theta = 0.1$, and target output $t$:

| Epoch | $x_1$ | $x_2$ | bias | $w_1$ | $w_2$ | $w_{\text{bias}}$ | $net$ | Output $y$ | Target $t$ | Action & Adjustm |
|---|---|---|---|---|---|---|---|---|---|---|
| **1** | 0 | 0 | 1 | 0.1 | 0.2 | -0.2 | -0.2 | 0 | 0 | Correct ( change. |
| **1** | 0 | 1 | 1 | 0.1 | 0.2 | -0.2 | 0.0 | 0 | 1 | **Incorrect** ). Update $w_1 := 0.1$ $w_2 := 0.2$ $w_{\text{bias}} := -$ |
| **1** | 1 | 0 | 1 | 0.1 | 1.2 | 0.8 | 0.9 | 1 | 1 | Correct ( change. |
| **1** | 1 | 1 | 1 | 0.1 | 1.2 | 0.8 | 2.1 | 1 | 1 | Correct ( change. |

*Note: After one complete epoch, the new weights are $w_1 = 1.1, w_2 = 1.2, w_{\text{bias}} = -0.2$ (following final updates at the start of Epoch 2).*

---

# 3. Single-Layer Perceptron using the Delta Rule

The Delta Rule (also known as the Adaline rule or Widrow-Hoff rule) updates the weights continuously based on the difference between the target and calculated output. Unlike the binary perceptron rule, it incorporates a **learning rate ($\eta$)** to control the step size of learning.

## Mathematical Formulation

The general weight update formula for input neuron $i$ mapping to output unit $\Omega$ is:

$$w_{i,\Omega} := w_{i,\Omega} + \eta \cdot o_i \cdot (t_\Omega - y_\Omega)$$

For a standard 2-input network with a bias:

$$w_1 := w_1 + \eta \cdot x_1 \cdot (t - y)$$

$$w_2 := w_2 + \eta \cdot x_2 \cdot (t - y)$$

$$w_{\text{bias}} := w_{\text{bias}} + \eta \cdot \text{bias} \cdot (t - y)$$

Where:

- $\eta$: Learning rate (e.g., $0.1$).
- $(t - y)$: The prediction error.

## Weight Update Trace Table (Delta Rule)

Given learning rate $\eta = 0.1$, initial weights $w_1 = 0.1, w_2 = 0.2, w_{\text{bias}} = -0.2$, inputs $x_1, x_2$, bias value $1$, and target output $t$:

| **Epoch** | $x_1$ | $x_2$ | **bias** | $w_1$ | $w_2$ | $w_{\text{bias}}$ | $net$ | **Output** $y$ | **Target** $t$ | **Action & Update** |
|---|---|---|---|---|---|---|---|---|---|---|
| **1** | 0 | 0 | 1 | 0.1 | 0.2 | -0.2 | -0.2 | 0 | 0 | Correct ($t$ |
| **1** | 0 | 1 | 1 | 0.1 | 0.2 | -0.2 | 0.0 | 0 | 1 | **Incorrect** weights: $w_1 := 0.1$ $w_2 := 0.2$ $w_{\text{bias}} := -$ |
| **1** | 1 | 0 | 1 | 0.1 | 0.3 | -0.1 | 0.0 | 0 | 1 | **Incorrect** weights: $w_1 := 0.1$ $w_2 := 0.3$ $w_{\text{bias}} := -$ |
| **1** | 1 | 1 | 1 | 0.2 | 0.3 | 0.0 | 0.5 | 1 | 1 | Correct ($t$ |